

ESP32C3SuperMini Chatgpt

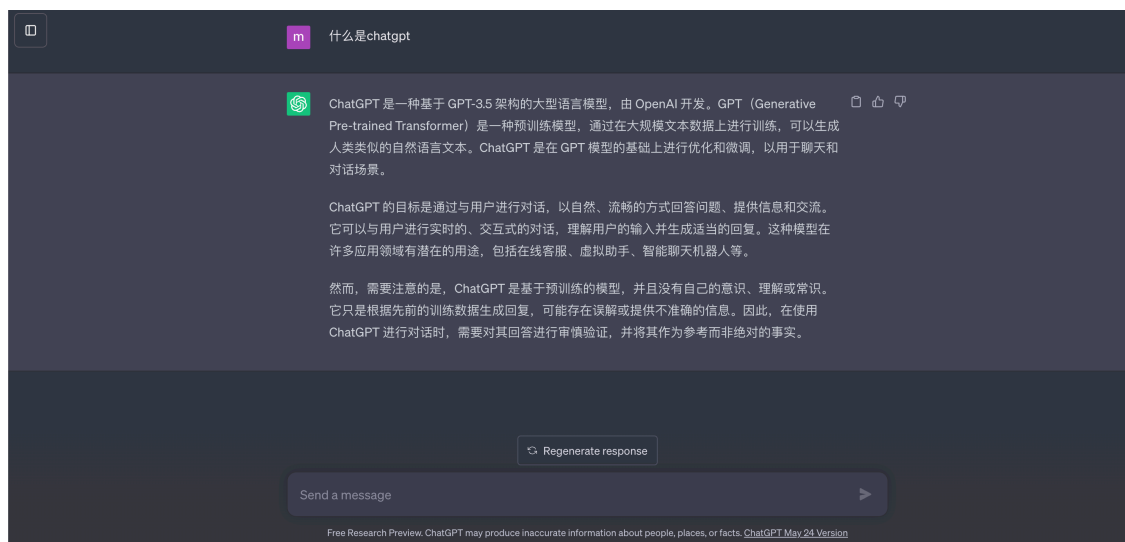
👤 [wmnologo](#) ⌚ 大约 14 分钟

学习在 ESP32C3 上使用 WiFiClient 和 HTTPClient - XIAO ESP32C3 和 ChatGPT 实际应用

概述

ChatGPT是人工智能研究实验室OpenAI于2022年11月30日发布的一种新型聊天机器人模型，是一种人工智能技术驱动的自然语言处理工具。

它能够通过学习 and 理解人类语言来进行对话，还可以根据聊天上下文进行交互，真正像人一样聊天和交流，甚至可以执行编写电子邮件、视频脚本、文案、翻译等任务。编码。



在嵌入式系统中，ChatGPT可以成为一个好帮手，帮助我们编写简单的程序，甚至检查和修复程序中出现的Bug。

令人兴奋的是，OpenAI官方提供了调用GPT-3.5模型的接口，这使得我们可以调用这些接口，并通过多种方法将这个伟大的模型部署到我们自己的嵌入式系统中。

ESP32C3SuperMini 是一款基于 Espressif ESP32-C3 WiFi/蓝牙双模芯片的 IoT 迷你开发板。它具有出色的射频性能，支持 IEEE 802.11 b/g/n WiFi 和蓝牙 5 (LE) 协议。可以完美支持 ESP32官方提供的WiFi Client和WiFi Server服务。当然，它能够完美支持Arduino。

因此，在本教程中，我们将引导用户学习和使用 ESP32C3 WiFiClient 和 HTTPClient 库、如何连接网络、如何发布网页以及 HTTP GET 和 POST 的基础知识。目标是调用 OpenAI ChatGPT 并创建您自己的问答网站。

入门

在本教程中，我们将使用 ESP32C3SuperMini 配置我们自己的 ChatGPT 问答页面。在此页面中，您可以输入您的问题，ESP32C3SuperMini 会记录您的问题，并使用 OpenAI 提供的 API 调用方法，使用 HTTP Client 发送请求命令，获取 ChatGPT 的答案并打印在串口中。

本教程中的任务可分为以下四个主要步骤。

- 配置 ESP32C3SuperMini 连接到网络：在这一步中，我们将学习使用 ESP32C3SuperMini 的基本 WiFi 配置流程，并了解 ESP32C3SuperMini 的基本操作，例如网络配置、连接网络服务和获取 IP 地址。
- 构建嵌入式网页：在这一步中，我们主要接触 WiFi 客户端库。通过使用该库的 GET 和 POST 函数，我们可以使用 HTML 编写自己的问答网页并将其部署在 ESP32C3SuperMini 之上。
- 通过内置网页提交问题：这一步我们主要学习如何使用 HTTP Client 中的 POST 方法按照 OpenAI API 标准来 POST 我们提出的问题。我们将主要关注如何从网页中收集和存储问题的过程。
- 从 ChatGPT 获取答案：在这一步中，我们学习在 HTTP 客户端中使用 POST 方法，并从返回的消息中提取我们需要的答案。最后一步就是梳理代码的结构并进行最后的集成。

材料

前期准备

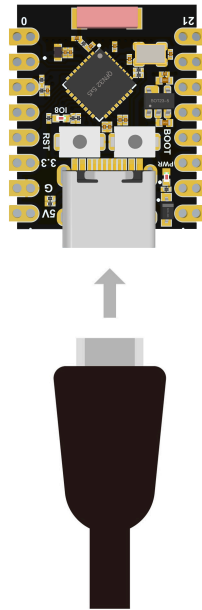
您需要准备以下内容：

- 1 个 ESP32C3SuperMini
- 1 台电脑
- 1 根 USB Type-C 数据线
- 1 个 ChatgptAPI（如果没有可以加群联系群主）

提示

有些USB线只能供电，不能传输数据。如果您没有 USB 线或者不知道您的 USB 线是否可以传输数据，可以购买[Type-c数据线](#) ↗

- 步骤 1.通过USB Type-C数据线将ESP32C3SuperMini连接到计算机



- 步骤1.根据您的操作系统下载并安装最新版本的Arduino IDE

如果下载缓慢可以在国内Arduino社区下载[ArduinoIDE下载地址](#) ↗

- 步骤 2.启动 Arduino 应用程序
- 步骤 3.将 ESP32 板包添加到 Arduino IDE

导航到File > Preferences , 然后使用以下 url 填写“Additional Boards Manager URL” :

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json ↗

导航到Tools > Board > Boards Manager...，在搜索框中输入关键字“esp32”，选择最新版本的esp32并安装它。

导航到工具 > 开发板 > ESP32 Arduino并选择“ESP32C3 Dev Module”。板的列表有点长，你需要滚动到底部才能到达它。

导航到“工具”>“端口”，然后选择所连接的 ESP32C3SuperMini 的串口名称。这可能是 COM3 或更高版本（COM1和COM2通常保留用于硬件串行端口）。

配置ESP32C3连接网络

ESP32SuperMini WiFi使用教程中有详细介绍。

当 ESP32 设置为 Wi-Fi 模式时，它可以连接到其他网络（例如路由器）。在这种情况下，路由器会为您的 ESP32开发板分配一个唯一的 IP 地址。

要使用 ESP32 Wi-Fi 功能，您需要做的第一件事就是在代码中包含 WiFi.h 库，如下所示：

```
1 #include <WiFi.h> js
```

要将 ESP32 连接到特定的 Wi-Fi 网络，您必须知道其 SSID 和密码。此外，该网络必须位于 ESP32 Wi-Fi 范围内。

首先，设置Wi-Fi模式。如果 ESP32 将连接到另一个网络（接入点/热点），则它必须处于站模式。

```
1 WiFi.mode(WIFI_STA); js
```

然后，用于WiFi.begin()连接到网络。您必须将网络 SSID 及其密码作为参数传递。

连接到 Wi-Fi 网络可能需要一段时间，因此我们通常添加一个 while 循环，不断检查连接是否已通过使用 建立WiFi.status()。当连接成功建立后，返回WL_CONNECTED。

当 ESP32 设置为 Wi-Fi 站时，它可以连接到其他网络（例如路由器）。在这种情况下，路由器会为您的 ESP32 板分配一个唯一的 IP 地址。要获取主板的 IP 地址，您需要 `WiFi.localIP()` 在与网络建立连接后进行调用。

```
1 void WiFiConnect(void){js  
2     WiFi.begin(ssid, password);  
3     Serial.print("Connecting to ");  
4     Serial.println(ssid);  
5  
6     while (WiFi.status() != WL_CONNECTED) {  
7         delay(500);  
8         Serial.print(".");  
9     }  
10  
11     Serial.println("");  
12     Serial.println("WiFi connected!");  
13     Serial.print("IP address: ");  
14     Serial.println(WiFi.localIP());  
15 }
```

变量 `password` 和 `ssid` 保存您要连接的网络的 密码 和 WIFI。

```
1 // Replace with your network credentialsjs  
2 const char* ssid = "REPLACE_WITH_YOUR_SSID";  
3 const char* password = "REPLACE_WITH_YOUR_PASSWORD";
```

这是一个非常简单的 WiFi 连接程序，将程序上传到 ESP32C3 SuperMini，然后打开串口助手，将波特率设置为 115200。如果连接顺利，你会看到打印出的 IP 地址。

如果您有兴趣阅读有关 ESP32C3 在 WiFi 中的应用和功能的更多信息，我们建议您阅读 [Random Nerd Tutorials ↗](#)。

构建嵌入式网页

ESP32 在 WiFi 库中集成了许多非常有用的 WiFiClient 函数，这使得我们可以在不添加额外库的情况下设计和开发嵌入式网页。

创建一个新的 WiFiServer 对象，以便使用该对象来控制 XIAO ESP32C3 建立的 IoT 服务器。

```
1 WiFiServer server(80);  
2 WiFiClient client1;
```

js

在上面的步骤中，我们让 ESP32C3SuperMini 连接到WiFi。WiFi连接成功后，您将可以从串口监视器中获取当前ESP32C3SuperMini的IP地址。此时，ESP32C3SuperMini已经成功搭建了Web服务器。您可以通过 ESP32C3SuperMini 的 IP 地址访问该 Web 服务器。

假设您的 ESP32C3SuperMini 的 IP 地址是192.168.7.152。然后你接下来可以通过浏览器输入这个IP地址。

输入这个IP地址后，我们可能只会看到一个空白页面。这是因为我们尚未发布该页面的页面内容。

现在让我们用 C 语言创建一个数组来存储我们想要布局的页面内容，即 HTML 代码。

```
1  const char html_page[] PROGMEM = {                                     js
2      "HTTP/1.1 200 OK\r\n"
3      "Content-Type: text/html\r\n"
4      "Connection: close\r\n" // the connection will be closed after
5      completion of the response
6      //"Refresh: 1\r\n"          // refresh the page automatically every n
7      sec
8      "\r\n"
9      "<!DOCTYPE HTML>\r\n"
10     "<html>\r\n"
11     "<head>\r\n"
12         "<meta charset=\"UTF-8\">\r\n"
13         "<title>Cloud Printer: ChatGPT</title>\r\n"
14         "<link rel=\"icon\"
15 href=\"https://files.seeedstudio.com/wiki/xiaoesp32c3-chatgpt/chatgpt-
16 logo.png\" type=\"image/x-icon\">\r\n"
17     "</head>\r\n"
18     "<body>\r\n"
19         "<p style=\"text-align:center;\">\r\n"
20         "<img alt=\"ChatGPT\"
21 src=\"https://files.seeedstudio.com/wiki/xiaoesp32c3-chatgpt/chatgpt-
22 logo.png\" height=\"200\" width=\"200\">\r\n"
23         "<h1 align=\"center\">Cloud Printer</h1>\r\n"
24         "<h1 align=\"center\">OpenAI ChatGPT</h1>\r\n"
25         "<div style=\"text-align:center;vertical-align:middle;\">"
26         "<form action=\"/\" method=\"post\">"
27         "<input type=\"text\" placeholder=\"Please enter your question\"
28 size=\"35\" name=\"chatgpttext\" required=\"required\"/>\r\n"
        "<input type=\"submit\" value=\"Submit\" style=\"height:30px;
width:80px;\"/>"
        "</form>"
        "</div>"
        "</p>\r\n"
        "</body>\r\n"
        "<html>\r\n"
    };
```

这段代码给我们带来了下图所示的页面效果。

提示

网页的 HTML 语法超出了本教程的范围。您可以自己学习使用 HTML，或者，我们也可以使用现有的生成工具来完成代码生成工作。我们建议使用[HTML 生成器](#)[↗]。值得注意的是，在 C 程序中，“\ ”和“””是特殊字符，如果想在程序中保留这些特殊字符的功能，需要在它们前面添加右斜杠。

Client1指的是Web服务器建立后的Socket客户端，下面的代码是Web服务器处理的流程。

```

1 client1 = server.available();
2 if (client1){
3     Serial.println("New Client.");           // print a message out the
4     serial port
5     // an http request ends with a blank line
6     boolean currentLineIsBlank = true;
7     while (client1.connected()){
8         if (client1.available()){ // Check if the client is connected
9             char c = client1.read();
10            json_String += c;
11            if (c == '\n' && currentLineIsBlank) {
12                dataStr = json_String.substring(0, 4);
13                Serial.println(dataStr);
14                if(dataStr == "GET "){
15                    client1.print(html_page); //Send the response body
16                to the client
17                }
18                else if(dataStr == "POST"){
19                    json_String = "";
20                    while(client1.available()){
21                        json_String += (char)client1.read();
22                    }
23                    Serial.println(json_String);
24                    dataStart = json_String.indexOf("chatgpttext=") +
25                    strlen("chatgpttext=");
26                    chatgpt_Q = json_String.substring(dataStart,
27                    json_String.length());
28                    client1.print(html_page);
29                    // close the connection:
30                    delay(10);
31                    client1.stop();
32                }
33                json_String = "";

```

```

34         break;
35     }
36     if (c == '\n') {
37         // you're starting a new line
38         currentLineIsBlank = true;
39     }
40     else if (c != '\r') {
41         // you've gotten a character on the current line
42         currentLineIsBlank = false;
43     }
44 }
45 }
46 }

```

在上面的示例程序中，我们需要使用`server.begin()`来启动 IoT 服务器。该语句需要放在函数中`setup`。

```

1  void setup()                                     js
2  {
3      Serial.begin(115200);
4
5      // Set WiFi to station mode and disconnect from an AP if it was
6      previously connected
7      WiFi.mode(WIFI_STA);
8      WiFi.disconnect();
9      while(!Serial);
10
11     Serial.println("WiFi Setup done!");
12     WiFiConnect();
13
14     // Start the TCP server server
15     server.begin();
16 }

```

一旦运行上述程序，并且在浏览器中输入 ESP32C3SuperMini 的 IP 地址（前提是您的主机也需要与 ESP32C3SuperMini 在同一 WIFI 中），则 WiFiClient 的 GET 步骤将开始执行。此时，我们借助客户端打印方法，提交页面的HTML代码。

```

1  if(dataStr == "GET "){                           js
2      client1.print(html_page);
3  }

```

我们在页面中设计了用于问题输入的输入框，当用户输入内容并点击提交按钮后，网页会获取按钮的状态并将输入的问题存储到字符串变量中chatgpt_Q。

```
1  json_String = "";
2  while(client1.available()){
3      json_String += (char)client1.read();
4  }
5  Serial.println(json_String);
6  dataStart = json_String.indexOf("chatgpttext=") + strlen("chatgpttext=");
7  chatgpt_Q = json_String.substring(dataStart, json_String.length());
8  client1.print(html_page);
9  // close the connection:
10 delay(10);
11 client1.stop();
```

运行效果如下图

提交问题发送给Chatgpt

在上一步的页面中，有一个输入框。输入框是我们需要用户输入他们想问的问题的地方。我们需要做的就是获取这个问题并通过OpenAI提供的API请求发送出去。

步骤1 注册 OpenAI 帐户。（国内无法正常注册，可以自行百度）

您可以[点击这里](#) 进入OpenAI的[注册地址](#) ↗。如果您之前已经注册过该帐户，则可以跳过此步骤。

步骤2。获取 OpenAI API。

[登录OpenAI网站](#) ↗，点击右上角您的账户头像，然后选择查看API密钥。

在新的弹出页面中选择+创建新密钥，然后复制您的密钥并保存。

同时，我们可以在程序中创建字符串变量并将此键复制到此处。

```
1 char chatgpt_token[] = "sk*****Rj9DYWSDFNA"; js
```

步骤3. 根据OpenAI的HTTP请求编写程序。

OpenAI提供了非常详细的[API使用教程](#)，以使用户可以使用自己的API密钥来调用ChatGPT。

根据ChatGPT的文档，我们需要发送请求的格式如下：

```
1 curl https://api.openai.com/v1/completions \
2 -H "Content-Type: application/json" \
3 -H "Authorization: Bearer YOUR_API_KEY" \
4 -d '{"model": "text-davinci-003", "prompt": "Say this is a test",
    "temperature": 0, "max_tokens": 7}' js
```

超文本传输协议 (HTTP) 用作客户端和服务端之间的请求-响应协议。GET用于从指定资源请求数据。它通常用于从 API 获取值。POST用于将数据发送到服务器以创建/更新资源。ESP32 可以使用三种不同类型的正文请求发出 HTTP POST 请求：URL 编码、JSON 对象或纯文本。这些是最常见的方法，应该与大多数 API 或 Web 服务集成。

以上信息非常重要，为编写HTTP POST程序提供了理论基础。首先，我们首先导入HTTPClient 库。

```
1 #include <HTTPClient.h> js
```

您还需要输入 OpenAI 域名，以便 ESP32C3 将问题发布到 ChatGPT。并且不要忘记 OpenAI API 密钥。

```
1 HTTPClient https;
2 const char* chatgpt_token = "YOUR_API_KEY";
3 char chatgpt_server[] = "https://api.openai.com/v1/completions"; js
```

我们需要使用 JSON 对象发出 HTTP POST 请求。

```
1 if (https.begin(chatgpt_server)) { // HTTPS
2     https.addHeader("Content-Type", "application/json");
3     String token_key = String("Bearer ") + chatgpt_token;
4     https.addHeader("Authorization", token_key);
5     String payload = String("{\"model\": \"text-davinci-003\",
6     \"prompt\": \"\"} + chatgpt_Q + String(\"\", \"temperature\": 0,
7     \"max_tokens\": 100}"); //Instead of TEXT as Payload, can be JSON as js
```

```

8   Payload
9   httpCode = https.POST(payload);    // start connection and send HTTP
10  header
11   payload = "";
12  }
   else {
       Serial.println("[HTTPS] Unable to connect");
       delay(1000);
   }

```

在程序中，我们通过POST()方法将payload 发送到服务器。chatgpt_Q是我们要发送到ChatGPT 的问题内容，该内容将在“获取问题”页面中提供。

如果您对 ESP32C3 HTTPClient 的更多功能感兴趣，我们建议您阅读[ESP32 HTTP GET 和 HTTP POST with Arduino IDE](#) ↗。

从chatgpt获取答案

接下来是整个教程的最后一步，我们如何获取ChatGPT的答案并记录下来。

我们继续阅读OpenAI提供的API文档来了解ChatGPT返回的消息内容的结构是什么样的。这将使我们能够编写程序来解析我们需要的内容。

```

1   {
2       "id": "cml-GERzeJQ4lvqPk8SkZu4XMIuR",
3       "object": "text_completion",
4       "created": 1586839808,
5       "model": "text-davinci-003",
6       "choices": [
7           {
8               "text": "\n\nThis is indeed a test",
9               "index": 0,
10              "logprobs": null,
11              "finish_reason": "length"
12          }
13      ],
14      "usage": {
15
16
17
18

```

```

19         "prompt_tokens": 5,
           "completion_tokens": 7,
           "total_tokens": 12
        }
    }

```

从OpenAI提供的参考文档中，我们知道接口返回的消息中问题答案的位置在{"choices": [{"text": "\n\nxxxxxxx"}]}.

所以现在我们可以确定我们需要的“答案”应该以\n\n开头并以,结尾。然后，在程序中，我们可以使用该方法检索文本开始和结束的位置indexOf()，并存储返回答案的内容。

```

1  dataStart = payload.indexOf("\\n\\n") + strlen("\\n\\n");
2  dataEnd = payload.indexOf(",", dataStart);
3  chatgpt_A = payload.substring(dataStart, dataEnd);

```

综上所述，我们可以使用 switch 方法结合程序当前的状态来确定我们应该执行程序的哪一步。

```

1  typedef enum
2  {
3      do_webserver_index,
4      send_chatgpt_request,
5      get_chatgpt_list,
6  }STATE_;
7
8  STATE_ currentState;
9
10 switch(currentState){
11     case do_webserver_index:
12         ...
13     case send_chatgpt_request:
14         ...
15     case get_chatgpt_list:
16         ...
17 }

```

至此，整个程序的逻辑就结构化了。点击下图即可获取完整的程序代码。请不要急于上传程序，您需要将程序的ssid、密码和chatgpt_token更改为您自己的。

▼ | 示例代码

```
1                                     js
2 // Load Wi-Fi library
3 #include "WiFi.h"
4 #include <HTTPClient.h>
5
6 // Replace with your network credentials
7 const char* ssid    = "WIFI账号";
8 const char* password = "WIFI密码";
9 //chatgpt api
10 const char* chatgpt_token = "有的openAPI-Key";
11
12 char chatgpt_server[] = "https://api.openai.com/v1/completions";
13
14 // Set web server port number to 80
15 WiFiServer server(80);
16 WiFiClient client1;
17
18 HTTPClient https;
19
20 String chatgpt_Q;
21 String chatgpt_A;
22 String json_String;
23 uint16_t dataStart = 0;
24 uint16_t dataEnd = 0;
25 String dataStr;
26 int httpCode = 0;
27
28 typedef enum
29 {
30     do_webserver_index,
31     send_chatgpt_request,
32     get_chatgpt_list,
33 }STATE_;
34
35 STATE_ currentState;
36
37 void WiFiConnect(void){
38     WiFi.begin(ssid, password);
39     Serial.print("Connecting to ");
40     Serial.println(ssid);
```

```

40     while (WiFi.status() != WL_CONNECTED) {
41         delay(500);
42         Serial.print(".");
43     }
44     Serial.println("");
45     Serial.println("WiFi connected!");
46     Serial.print("IP address: ");
47     Serial.println(WiFi.localIP());
48     currentState = do_webserver_index;
49 }
50
51 const char html_page[] PROGMEM = {
52     "HTTP/1.1 200 OK\r\n"
53     "Content-Type: text/html\r\n"
54     "Connection: close\r\n" // the connection will be closed after
55     completion of the response
56     //"Refresh: 1\r\n" // refresh the page automatically
57     every n sec
58     "\r\n"
59     "<!DOCTYPE HTML>\r\n"
60     "<html>\r\n"
61     "<head>\r\n"
62         "<meta charset='UTF-8'>\r\n"
63         "<title>Cloud Printer: ChatGPT</title>\r\n"
64         "<link rel='icon'"
65         href="https://files.seeedstudio.com/wiki/xiaoesp32c3-
66         chatgpt/chatgpt-logo.png" type="image/x-icon">\r\n"
67         "</head>\r\n"
68         "<body>\r\n"
69         "<p style='text-align:center;'>\r\n"
70         "\r\n"
73         "<h1 align='center'>Cloud Printer</h1>\r\n"
74         "<h1 align='center'>OpenAI ChatGPT</h1>\r\n"
75         "<div style='text-align:center;vertical-align:middle;'"
76         "<form action='/' method='post'"
77         "<input type='text' placeholder='Please enter your question'"
78         size="35" name="chatgpttext" required="required"/>\r\n"
79         "<input type='submit' value='Submit' style='height:30px;
80         width:80px;'"
81         "</form>"
82         "</div>"

```



```

83     "</p>\r\n"
84     "</body>\r\n"
85     "<html>\r\n"
86 };
87
88 void setup()
89 {
90     Serial.begin(115200);
91
92     // Set WiFi to station mode and disconnect from an AP if it was
93     previously connected
94     WiFi.mode(WIFI_STA);
95     WiFi.disconnect();
96     while(!Serial);
97
98     Serial.println("WiFi Setup done!");
99
100    WiFiConnect();
101    // Start the TCP server server
102    server.begin();
103 }
104
105 void loop()
106 {
107     switch(currentState){
108         case do_webserver_index:
109             Serial.println("Web Production Task Launch");
110             client1 = server.available();           // Check if the
111             client is connected
112             if (client1){
113                 Serial.println("New Client.");       // print a message
114                 out the serial port
115                 // an http request ends with a blank line
116                 boolean currentLineIsBlank = true;
117                 while (client1.connected()){
118                     if (client1.available()){
119                         char c = client1.read();     // Read the rest of
120                         the content, used to clear the cache
121                         json_String += c;
122                         if (c == '\n' && currentLineIsBlank) {
123                             dataStr = json_String.substring(0, 4);
124                             Serial.println(dataStr);
125                             if(dataStr == "GET "){

```

```

126         client1.print(html_page);           //Send the response
127     body to the client
128     }
129     else if(dataStr == "POST"){
130         json_String = "";
131         while(client1.available()){
132             json_String += (char)client1.read();
133         }
134         Serial.println(json_String);
135         dataStart = json_String.indexOf("chatgpttext=") +
136 strlen("chatgpttext="); // parse the request for the following
137 content
138         chatgpt_Q = json_String.substring(dataStart,
139 json_String.length());
140         client1.print(html_page);
141         Serial.print("Your Question is: ");
142         Serial.println(chatgpt_Q);
143         // close the connection:
144         delay(10);
145         client1.stop();
146         currentState = send_chatgpt_request;
147     }
148     json_String = "";
149     break;
150 }
151 if (c == '\n') {
152     // you're starting a new line
153     currentLineIsBlank = true;
154 }
155 else if (c != '\r') {
156     // you've gotten a character on the current line
157     currentLineIsBlank = false;
158 }
159 }
160 }
161 }
162 delay(1000);
163 break;
164 case send_chatgpt_request:
165     Serial.println("Ask ChatGPT a Question Task Launch");
166     if (https.begin(chatgpt_server)) { // HTTPS
167         https.addHeader("Content-Type", "application/json");
168         String token_key = String("Bearer ") + chatgpt_token;

```

```

169         https.addHeader("Authorization", token_key);
170         String payload = String("{\"model\": \"text-davinci-003\",
171 \"prompt\": \"\"} + chatgpt_Q + String("\", \"temperature\": 0,
172 \"max_tokens\": 100}"); //Instead of TEXT as Payload, can be JSON as
173 Payload
174         httpCode = https.POST(payload);    // start connection and
175 send HTTP header
176         payload = "";
177         currentState = get_chatgpt_list;
178     }
179     else {
180         Serial.println("[HTTPS] Unable to connect");
181         delay(1000);
182     }
183     break;
184 case get_chatgpt_list:
185     Serial.println("Get ChatGPT Answers Task Launch");
186     // httpCode will be negative on error
187     // file found at server
188     if (httpCode == HTTP_CODE_OK || httpCode ==
HTTP_CODE_MOVED_PERMANENTLY) {
        String payload = https.getString();
//        Serial.println(payload);
        dataStart = payload.indexOf("\\n\\n") + strlen("\\n\\n");
        dataEnd = payload.indexOf("\",\"", dataStart);
        chatgpt_A = payload.substring(dataStart, dataEnd);
        Serial.print("ChatGPT Answer is: ");
        Serial.println(chatgpt_A);
        Serial.println("Wait 10s before next round...");
        currentState = do_webserver_index;
    }
    else {
        Serial.printf("[HTTPS] GET... failed, error: %s\\n",
https.errorToString(httpCode).c_str());
        while(1);
    }
    https.end();
    delay(10000);
    break;
}
}

```

本代码来源于[代码地址](#) ↗